ROBUST PREDICTION OF ANTIMICROBIAL RESISTANCE IN *E. coli* FROM GENOMIC k-MER PROFILES

Mateo Mangialomini
Polytechnique Montréal
mateo.mangialomini@polymtl.ca

Adrien Marie Saouma
Polytechnique Montréal
saoumadrien@gmail.com

Maxime Tran
Polytechnique Montréal
maxime-anh-duy.tran@etud.polymtl.ca

ABSTRACT

We predict whether an *E. coli* isolate is resistant or susceptible to an antibiotic from genome-derived k-mer counts (about one million features for 1 939 isolates) and isolate metadata. Our pipeline combines variance filtering, TF-IDF weighting and truncated SVD to compress the k-mer profiles into 1 000 components, which are concatenated with the encoded metadata. Because the private test set follows a phylogenetic split, we favored robustness over public-leaderboard performance: a weighted ensemble of LightGBM, CatBoost and elastic-net logistic regression, with weights fitted on out-of-fold predictions, reached a macro F1 score of 0.752 on the private leaderboard and reduced the public/private gap from 0.11 to 0.024.

1 INTRODUCTION

Antimicrobial resistance (AMR) is recognized by the World Health Organization as one of the top ten global public health threats. The rapid emergence of resistant bacterial strains, particularly in *Escherichia coli*, compromises the effectiveness of standard antibiotic treatments. While whole-genome sequencing has become routine, translating raw genomic data into accurate predictions of phenotypic resistance remains a major bioinformatics challenge.

This Kaggle competition provides a real-world classification task: predict whether an *E. coli* isolate is resistant or susceptible to a specific antibiotic using pre-computed k-mer count matrices derived from its genome, supplemented by rich metadata (geographical origin, isolation source, collection date, etc.). The evaluation metric is the macro-averaged F1 score, which appropriately handles the expected class imbalance.

In this work, we propose a robust model combining a TF-IDF transformation of high-dimensional k-mer profiles, variance-based and SVD feature selection, integration of metadata features, and an ensemble of XGBoost, CatBoost, LightGBM, random forest and regularized logistic regression models.

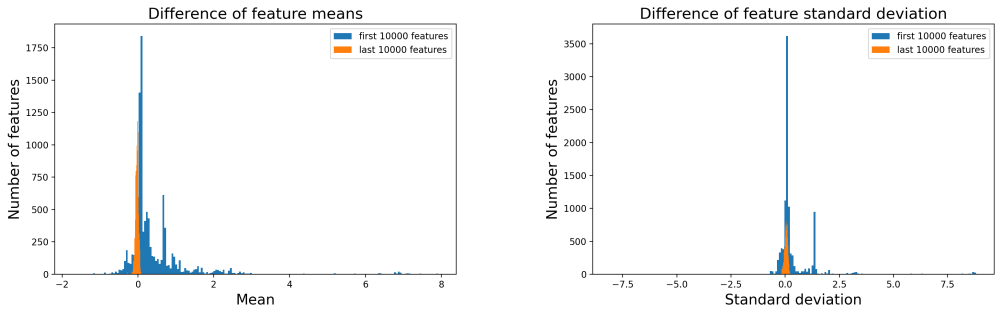
2 FEATURE SELECTION

The training set is composed of 1 939 samples with one million features. To avoid the curse of dimensionality and to reduce computational cost, we reduced the dimensionality of the k-mer data. We also expected the metadata to be useful for AMR detection.

2.1 VARIANCE

First, we reduced the number of k-mers to 100 000 using variance selection. We believe that k-mers with low variance have little impact on the classification. On the one hand, k-mers present or absent in most genomes have low variance and carry no information about resistance mutations. On the other hand, k-mers linked to resistance mechanisms show higher variance.

Figure 1 compares the between-class differences in means and standard deviations for the 10 000 highest-variance and the 10 000 lowest-variance features. The high-variance features show a positive skew in mean differences (up to 8), indicating significant shifts in average values across classes. In contrast, the low-variance k-mers cluster around zero, showing low discriminative power. The standard deviation differences follow a similar trend, with high-variance features displaying more class-specific variability than low-variance ones. This analysis confirms that variance filtering retains most of the information while greatly reducing the number of features.



(a) Means

(b) Standard deviations

Figure 1: Histograms of the differences between class 1 and class 0 for the 10 000 highest-variance and the 10 000 lowest-variance k-mers.

2.2 TF-IDF Transformation

To reduce the impact of genome length variation and to emphasize the k-mers that truly matter for the prediction, we applied a TF-IDF (term frequency–inverse document frequency) normalization. In this formulation, each genome is treated as a "document" and each k-mer as a "term" within a shared vocabulary. We first describe how this normalization works and then its impact on the model.

For k-mer j, let c_{ij} denote its count in genome i. The term frequency is

$$\mathrm{tf}_{ij} = \frac{c_{ij}}{\sum_{j'} c_{ij'}},$$

which converts absolute counts into relative frequencies and prevents longer genomes from dominating simply because of larger total k-mer counts. Let N be the number of genomes in the training set and df_j the number of genomes containing k-mer j. The inverse document frequency is

$$\mathrm{idf}_j = \log\left(\frac{1+N}{1+\mathrm{df}_j}\right) + 1,$$

which downweights k-mers that are widespread across genomes and therefore less informative. The final TF-IDF weight is

$$\mathrm{tfidf}_{ij} = \mathrm{tf}_{ij} \cdot \mathrm{idf}_j,$$

which emphasizes k-mers that are frequent within a specific genome but rare across genomes. The point was to give lower weights to k-mers present in most genomes, as they probably represent core sequences that do not trigger resistance when mutated, and higher weights to those that are rare across genomes but frequent in some of them.

To support the use of this normalization, we ran an ablation study in which the model was trained once with TF-IDF and once without it (Table 1).

With TF-IDF, the F1 score on the public test set increased by approximately 3.5%, showing its importance. On the private set, however, it decreased slightly, by 0.55%. One hypothesis is that the transformation correctly highlighted the discriminative k-mers in the training data but overspecialized on them: because the test set follows a phylogenetic split, it contains genomes unlike those seen in training. While designing this transformation, we focused too much on the ablation results and did not anticipate its impact on the robustness of our model; we relied on the ensemble (Section 3) to provide that robustness. Moreover, we used the default TF-IDF settings; tuning sublinear TF scaling, IDF smoothing and vector normalization could have improved performance.

Table 1: Ablation study of the TF-IDF transformation (macro F1 score).

TF-IDF	Public F1	Private F1
Excluded	0.76222	0.72490
Included	0.78890	0.72093

2.3 Truncated SVD

Even after variance filtering and TF-IDF, the dataset still contains far more features (100 000) than samples (1 939). Many of these k-mers are also correlated, which enlarges the feature space without adding much useful information and can affect how the model learns.

Truncated SVD addresses these issues by reducing the dimensionality of the TF-IDF matrix. Unlike PCA, it does not center the data before computing the decomposition, which makes it suitable for large sparse matrices like ours.

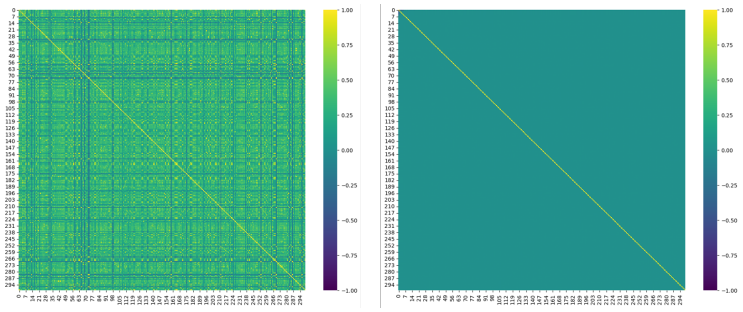


Figure 2: Correlation matrices of a random subset of 300 features of the TF-IDF matrix before (left) and after (right) truncated SVD. The features differ between the two panels, and so do the subsets.

The left panel of Figure 2 shows the correlation structure in a random subset of 300 k-mers. Several groups of features are strongly correlated, illustrating the redundancy present in the original matrix.

We keep 1 000 components after truncated SVD. These components are orthogonal, meaning that they are uncorrelated and capture distinct directions of variation in the data, as shown in the right panel of Figure 2. This reduces redundancy and gives the model a more compact representation to work with.

2.4 Metadata

The provided metadata contains six categorical fields (organism group, isolation type, location, isolation source, laboratory typing platform and testing standard) and a collection timestamp (creation date). Missing categorical values were explicitly encoded as a dedicated "Unknown" category to avoid information loss. The creation date was converted into a numerical feature, "days since reference date", using 2010-01-01 as the reference (it predates all samples), and then standardized. All categorical variables were one-hot encoded.

We believe that these data are important and can affect the model. Different regions expose bacteria to different environmental factors, such as temperature, UV radiation and mutagens. Depending on these factors, mutations may be more frequent and thus affect AMR.

After all this processing, we are left with a matrix of 1 939 samples and 1 105 features.

3 Ensemble Method

We chose an ensemble model for this competition because we were primarily focusing on robustness. We wanted to combine the strengths of the models we trained, in the hope that each would fill the gaps left by the others.

3.1 Models

3.1.1 Design Considerations

While developing our model, we focused primarily on robustness. Our first model performed well on the public set (XGBoost: 0.80771). However, we expected it to perform worse on the private set because of the phylogenetic split: some patterns appearing in the training data do not show up in the private set and vice versa. This assumption turned out to be correct, as our first model overfitted the training data and performed poorly on the private set (0.69467). This is one of the reasons why we added logistic regression on top of the gradient boosting models (Section 3.1.2). Combining different types of models and averaging their predictions (Section 3.2) gave a more balanced and robust final prediction. We also relied on strong regularization to further reduce overfitting.

Another important point was the class imbalance. The training set contains 1 666 samples of class 0 and only 273 of class 1, a ratio of more than 6 to 1. To deal with this, we used stratified cross-validation so that each fold keeps the same class proportions as the full dataset. Moreover, we used class weights to give more weight to the minority class.

The competition metric is the F1 score rather than accuracy, most likely because of this imbalance: on such a skewed dataset, a model can reach a high accuracy by mostly predicting the majority class. This is what we observed during cross-validation, with accuracies around 0.87 that do not reflect the difficulty of predicting the resistant class.

3.1.2 Model Choice

XGBoost, CatBoost and LightGBM XGBoost, CatBoost and LightGBM are among the most widely used gradient boosting frameworks in machine learning and have performed very well in past competitions. They handle sparse, high-dimensional data well and offer built-in regularization and robust handling of imbalanced datasets, which makes them well suited to capturing nonlinear interactions among the k-mers.

Random forest Random forest (`scikit-learn`) is an ensemble of decision trees that adds robustness to our model.

Logistic regression Logistic regression works well as a secondary voter: it handles high-dimensional data well, and regularization helps avoid overfitting. Moreover, with balanced class weights, it addresses the imbalance between resistant and susceptible samples by penalizing errors on the minority class more heavily. As a linear model, it adds diversity to the nonlinear boosted trees, reducing correlated errors in the soft-voting setup. Its main role is therefore to make the ensemble more robust and stable.

3.2 Weighted Vote

The goal of having multiple models is to leverage their complementary strengths. Rather than relying on a simple majority vote, we combined the predictions of our models through a weighted average. Our first submissions showed that the boosting frameworks (XGBoost, LightGBM, CatBoost) could independently achieve good performance on the public leaderboard, so the main purpose of adding random forest and logistic regression was to improve the predictions in cases where boosting failed.

To determine the optimal contribution of each model, we optimized the weights on cross-validated out-of-fold predictions (4 folds): the predictions on the validation folds estimate each model's performance, and `scipy.optimize` computes the optimal weights (Table 2). Surprisingly, XGBoost was completely outperformed by LightGBM and CatBoost, which use slightly different boosting methods and optimizations. Random forest is probably too simple compared with the boosting methods and captures nothing they miss. Logistic regression, on the other hand, fulfils its role: although it receives a small weight, it provides a small improvement where boosting is less effective.

Our initial approach had been to set the weights manually, giving more importance to XGBoost because of its good public F1 score in our first submissions. The optimized weights in Table 2 show that this would have biased the ensemble and lowered its performance.

Table 2: Ensemble weights, optimized on cross-validated out-of-fold F1 scores.

Model	Weight
XGBoost	0.0000
LightGBM	0.4692
CatBoost	0.4560
Random forest	0.0000
Logistic regression	0.0747

3.3 Random Search

For hyperparameter optimization, we chose random search over exhaustive grid search: it explores the hyperparameter space more broadly without spending many trials on hyperparameters that have little impact on the score.

4 Results

4.1 Hyperparameter Tuning

We tuned the five candidate models independently with `RandomizedSearchCV` (50 trials, 4-fold stratified cross-validation). The search spaces deliberately spanned both high-capacity and strongly regularized configurations. Table 3 reports the best configuration found for each model.

Table 3: Best hyperparameters found by `RandomizedSearchCV`. Where a row lists two names, the second is the CatBoost parameter. XGBoost and random forest had clearly lower cross-validation F1 scores and received zero weight in the final ensemble.

Parameter	LightGBM	CatBoost	XGBoost	Random forest	LogReg
learning rate	0.0213	0.164	0.0133	–	–
max depth	6	3	5	5	–
n estimators / iterations	291	246	247	282	–
num leaves	79	–	–	–	–
subsample / bagging temp.	0.810	0.577	0.876	–	–
colsample bytree / border count	0.860	223	0.836	–	–
min child weight / L2 leaf reg.	10.39	6.72	3	–	–
reg. alpha	0.183	–	0.645	–	–
reg. lambda	0.608	–	0.349	–	–
gamma	–	–	0.203	–	–
max features	–	–	–	sqrt	–
bootstrap	–	–	–	True	–
C	–	–	–	–	6.83
L1 ratio	–	–	–	–	0.61
penalty	–	–	–	–	elasticnet

The search confirmed the dominance of LightGBM and CatBoost, both converging toward low learning rates and strong regularization. The optimal CatBoost configuration is remarkably shallow (depth 3), indicating that the TF-IDF and SVD representation is already highly discriminative. Logistic regression consistently favored elastic-net regularization with an L1 ratio of approximately 0.61, suggesting that sparse feature selection remains beneficial after dimensionality reduction. Despite extensive tuning, XGBoost and random forest lagged behind in internal validation and contributed nothing to the final prediction. The retained models were retrained on the full training set with the hyperparameters above before the weighted ensemble was built.

4.2 Score Evolution

Figure 3 shows the macro F1 scores on the public and private leaderboards across our eleven submissions, in chronological order. During the competition, only public scores were available.

Our first submissions, based on a single XGBoost model trained on variance-selected k-mers and metadata, quickly exceeded 0.80 on the public leaderboard. However, we strongly suspected severe overfitting: a model that learns spurious correlations present in a random split would inevitably fail on phylogenetically distant isolates. Rather than continuing to optimize for the public score, we deliberately sacrificed it when necessary to improve robustness.

We therefore systematically favored stronger regularization, model diversity (introduction of LightGBM and CatBoost) and conservative blending, even when these changes temporarily lowered the public score, sometimes by more than 4 points. One particularly telling decision was to test a strongly regularized version without TF-IDF: despite a public score of only 0.761, we submitted it because we believed it would reduce specialization to the training distribution. In the final phase, an automated weighting of out-of-fold predictions decided the ensemble composition: XGBoost and random forest received zero weight, while LightGBM, CatBoost and a small contribution from logistic regression formed the final model.

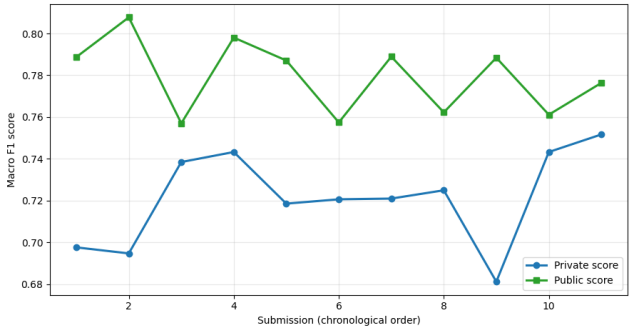


Figure 3: Public (green) and private (blue) macro F1 scores over the eleven submissions. Only public scores were visible during the competition.

The private leaderboard, revealed only at the end, validated this strategy: our final submission achieved a macro F1 score of 0.776 on the public leaderboard and 0.752 on the private one, a gain of more than 5.5 points on the phylogenetic split compared with our initial baseline, and the public/private gap shrank from over 0.11 to 0.024. This confirms that our focus on robustness, even at the cost of short-term public performance, was the right approach for this phylogenetically challenging task.

5 Discussion

The private leaderboard results indicate that our models generalize to unseen bacteria from a different evolutionary branch. The main area of improvement would be a more diverse ensemble. Using random forest alongside boosting methods is redundant; we should have focused on radically different models that could represent the data differently, such as support vector machines or neural networks. For the preprocessing pipeline, we could have explored other feature selection methods, such as Boruta, mutual information or chi-squared tests, which might have yielded limited improvements.

6 Statement of Contribution

We hereby state that all the work presented in this report is that of the authors. Mateo mainly focused on the metadata, data loading and exploration, and running the code. Adrien engineered the feature selection and Maxime created the meta-model. We all collaborated on model choice and tuning.